Verifiable Peephole Optimization for the Execution Layer of Blockchain Information Systems: dMIR Carry-Chain Operator Extension and SMT Equivalence Checking

Anonymous Author

Anonymous Institution

Abstract. Blockchain JIT peephole rewrites must preserve transaction semantics bit-for-bit; unverified rules risk consensus forks. General-purpose intermediate representations such as LLVM IR carry bit-level side-products (carry, overflow) through multi-return tuples. A single rewrite rule therefore spans several IR nodes and cannot be stated as a single verifiable unit in Alive/Alive2. Engineers consequently choose among dropping the rule, deploying it unverified, or proving it on paper. dMIR (Deterministic Middle IR), the middle IR of DTVM (Deterministic Virtual Machine), is extended with IR-side first-class bit-vector operators: bit-level patterns are modeled as dMIR first-class operators without modifying the solver, so the resulting rules enter the Z3 bit-vector equivalence check. A two-tier verifiable peephole rewrite system is built on this extension, with 70 dMIR rules checked rule-by-rule by Z3, 13 x86 mechanical-redundancy rules covered by a matched test suite, and a synthesis pipeline enumerating 853 candidates. The 27 evmone-bench workloads show a 7.24% geometric-mean speedup with a 25.79% peak; on 5884 Cancun differential tests, account state, storage, and gas consumption are byte-identical between the enabled and disabled sides.

Keywords: Blockchain information systems · EVM JIT · dMIR · Peephole optimization · SMT equivalence checking · Carry chain

1 Introduction

The execution layer of a blockchain information system is, in essence, a replicated consensus state machine: every node on the consensus path must produce bit-identical results for the same transaction. The EVM is the most widely deployed smart-contract execution environment, and its just-in-time (JIT) compiler is progressively replacing the interpreter in mainstream clients, becoming a core component on the consensus-critical path. Peephole rewrite rules in the JIT act directly on the machine-code generation stage, so their correctness bears directly on the determinism of on-chain execution.

If the equivalence judgment of a peephole rewrite rule is wrong and the rule is deployed on only some clients, the nodes may produce inconsistent execution results and trigger a chain fork—one of the most expensive failure modes

in a blockchain information system. Existing formal verification work on EVM peephole rewrites focuses mostly on the bytecode block-level abstraction. The machine-level patterns emitted by a JIT compiler during code generation—especially the multi-word carry chains that arise when the 256-bit word length is lowered onto 64-bit hardware—sit at an adjacent but different abstraction layer, and an end-to-end automatically verifiable rule set with a concrete deployment path is still missing.

The root cause of this gap is how intermediate representations model the bit-level side-products of arithmetic (such as the carry bit). In a general-purpose intermediate representation such as LLVM IR, the basic arithmetic operations are defined only on N-bit integers; the carry bit must be retrieved indirectly through an intrinsic that returns a multi-return tuple of {result, carry}. The bit-level side-product is not a native type. Rules that depend on such side-products can therefore only be expressed through indirect structures in Alive [5] and similar automatic equivalence-checking tools; the bottleneck is not the capacity of the SMT solver [6] but the expressiveness of the IR and its pattern matching. For a consensus-critical JIT, if a rule class cannot be verified automatically, the engineer is left with three choices: drop the rule, ship it without formal verification, or rely on a hand-written paper proof. Each choice sacrifices performance, endangers consensus, or fails to scale. The result is that such rules are systematically excluded from the optimization surface.

The direction taken in this paper is the following: *without modifying the solver and without loosening the differential-test threshold, expand the coverage of SMT-verifiable peephole rules by extending the IR with first-class bit-vector operators.* In dMIR, the bit-level patterns that previously required an indirect multi-return encoding become first-class operators, so the corresponding rules fall naturally within the decidable fragment of the Z3 bit-vector equivalence check.

Contributions. The contributions of this paper are as follows:

1. **An automatically verifiable peephole system through IR-side extension.** Bit-level side-products that a general-purpose IR can only carry through a multi-return tuple become first-class bit-vector operators of dMIR. Without modifying the solver and without loosening the differential-test threshold, the IR extension moves a rule class that previously sat outside the Alive/Alive2 automatic equivalence-verification framework into the decidable fragment of the Z3 bit-vector equivalence check [6]. A two-tier rewrite system is built on **70** dMIR production rules (5 of which rely on the first-class modeling and cannot be expressed as a single peephole rule on general IR) together with **13** rules at the x86 code-generation layer.
2. **Capacity and admission-rate characterization of the synthesis pipeline.** **853** candidates are evaluated under the same Z3 admission check, producing a single capacity-and-false-miss measurement. The admission rates on the two enumeration paths are **98.7%** for algebraic enumeration and **21.8%** for hand-constructed carry templates. The synthesis pipeline is independent of

the production rule set and its results serve as a Z3 admission-rate reference for future rule expansion.

3. **End-to-end performance and correctness.** Across the **27** EVM benchmarks of evmone-bench, the geometric-mean speedup is **+7.24%** (95% confidence interval [+2.59%, +12.11%], lower bound significantly greater than zero), with a peak of **+25.79%**; **5884** differential tests agree byte-for-byte on state and gas between the rules-enabled and rules-disabled sides.

Relative to the EVM bytecode block-level formal rule set of Albert et al. [1] at CAV 2023, the rule set in this paper operates at the JIT intermediate representation and machine-code layers that the bytecode abstraction cannot reach (quantitative comparison in §5).

2 Background and Motivation

2.1 Hardware Cost and Carry-Chain Shape of U256 Arithmetic

The native word length of the EVM is 256 bits, whereas general-purpose registers on current commodity hardware are 64 bits wide. An EVM-level U256 addition expands, after JIT code generation, into four machine-level additions: one ordinary 64-bit `add` followed by three carry-propagating `adc` instructions that form a *serial carry chain*. Subtraction is analogous, using one `sub` and three `sbb` instructions. The listing below shows the 4-limb code on both sides for a typical U256 addition:

dMIR (4-limb addition)	x86-64 output
`r0 = add(a0, b0)`	`add rax, rcx`
`r1 = ADC(a1, b1, cout0)`	`adc rbx, rdx`
`r2 = ADC(a2, b2, cout1)`	`adc rsi, rdi`
`r3 = ADC(a3, b3, cout2)`	`adc r8, r9`

Among all instructions emitted by the JIT, instructions directly tied to U256 multi-word arithmetic account for about **2.15%** (a weighted mean across 14 U256-heavy benchmarks; these 14 are the U256-heavy subset of the 27 benchmarks in §4, and the per-benchmark distribution is very uneven—see Table 4 in §4). The share is small, but the carry chain has a special property: every `adc`/`sbb` on the chain depends on the carry output of the previous instruction, forming a strict *serial data dependency* that instruction-level parallelism cannot hide. U256 instructions are therefore few in number yet contribute disproportionately to execution time. The family-attribution experiment in §4 (Table 4) shows that the hit rate of U256-related rules on U256-heavy benchmarks ranges from 25% to 99.9% with a very uneven distribution, indicating that the carry-chain pattern dominates those contracts. On non-U256-heavy benchmarks such as the sha1 family and `jump_around`, the U256 hit rate is well below that range, and the performance gain comes mainly from the mechanical-redundancy cleanup at the x86 layer.

Table 1. Four typical carry-chain rules that Alive2 (based on LLVM IR) cannot directly verify for equivalence because of the multi-return tuple encoding. The first three are single `ADC/SBB` rules where the right-hand side drops the carry output; the fourth is a 4-limb carry-chain merge rule.

Rule	Rewrite
`adc_zero_carry`	$\texttt{ADC}(x, y, 0) \to \texttt{add}(x, y)$
`sbb_zero_borrow`	$\texttt{SBB}(x, y, 0) \to \texttt{sub}(x, y)$
`sbb_self_zero`	$\texttt{SBB}(x, x, 0) \to 0$
4-limb ADC chain	$4 \times$ `uadd.w.overflow` $\to$ `add.i256`

2.2 Expressiveness Limit of General-Purpose IR on the Carry Chain

In a general-purpose compiler IR such as LLVM, the basic arithmetic operations (`add`, `sub`, `mul`) are defined only on N-bit integers `iN`. To obtain the carry bit explicitly, the programmer must call the intrinsic `@llvm.uadd.with.overflow.iN`, which returns a `{iN, i1}` multi-return tuple—the result and the carry bit are packed into the same structure. The carry bit is therefore *not a native type* in LLVM IR but is hidden in the second field of a struct.

This design limits the expressiveness of automatic equivalence checkers. Following the peephole verification framework proposed by Lopes et al. in Alive [5] and inherited by Alive2 [4], Table 1 lists four typical carry-chain rules that Alive2 cannot directly verify for equivalence—Alive2 returns a type mismatch or reports an undecidable outcome. Under the Alive/Alive2 framework these four rules fall in the **not automatically verifiable** category; the limitation stems from how the general-purpose IR models the carry bit and is independent of the rule implementation and the solver capacity.

The Alive2 command lines, failure messages, and version numbers (2026-03-20 mainline `v21.0-78-g28b9f9c9`, Z3 4.8.12) for all four rules are archived in the artifact (§6); the relation to Hydra [7] (OOPSLA 2024, a different system from the Hydra Framework at USENIX Security 2018 cited in §5) is discussed in §5.

The bottleneck is not the capacity of the SMT solver [6]. The remaining six U256 rules of the same class, which do not involve multi-return carry propagation, all pass Alive2 equivalence checking with an average runtime of about 60 ms. The 256-bit bit-vector size is not an obstacle for the solver; the bottleneck lies in how the IR expresses the carry bit.

Figure 1 contrasts the two IRs structurally on carry-propagating addition: the LLVM IR on the left needs five lines of code to call `uadd.with.overflow` and extract the result and the carry bit from the multi-return tuple, whereas on the right a single `ADC` instruction in dMIR suffices. If the IR natively supports the carry bit, the four rules above fall within the scope of automatic equivalence checking. dMIR targets the EVM JIT and does not aim at generality: it trades direct interoperability with LLVM for automatic SMT verification of carry-chain

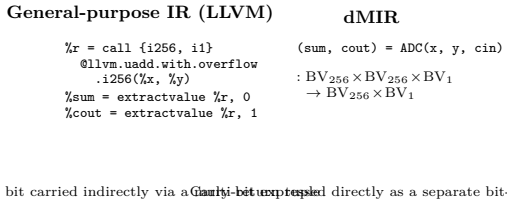

Fig. 1. Expressiveness comparison of general-purpose IR and dMIR on carry-propagating addition.

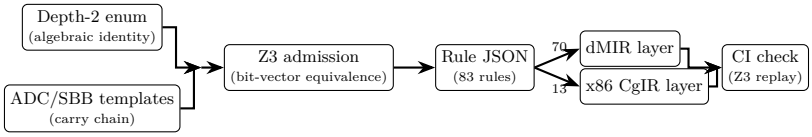

Fig. 2. Offline rule synthesis and two-tier integration pipeline.

rules. The four rules in Table 1 cannot be verified for equivalence directly on a general-purpose IR, but Z3 discharges them directly under the dMIR bit-vector model.

3 SMT-Verifiable Peephole Rewrite Rule Synthesis Based on dMIR

Figure 2 shows the full offline rule synthesis and two-tier integration pipeline. This section is organized in three parts: the dMIR bit-vector semantic model (§3.1), candidate enumeration and the SMT admission check (§3.2), and the layered design of the two-tier rule set (§3.3).

3.1 dMIR Bit-Vector Semantics and Carry-Chain Operators

The core data type of dMIR is a parametric-width bit-vector BV_n. The specialization BV_1 carries the carry input (c_{in}) and the carry output (c_{out}). The semantics of the carry-propagating addition `ADC` is defined as

$$\texttt{ADC} : \mathrm{BV}_n \times \mathrm{BV}_n \times \mathrm{BV}_1 \;\rightarrow\; \mathrm{BV}_n \times \mathrm{BV}_1,$$

that is, two n-bit operands and a 1-bit carry input map to an n-bit result and a 1-bit carry output. The borrow-propagating subtraction `SBB` has a fully symmetric signature.

In Z3 [6], the above carry propagation has a natural expression through bit-vector concatenation and zero extension. The following listing shows the Z3 encoding of `ADC`:

```
x, y = BitVecs('x y', 256)
cin = BitVec('cin', 1)
wide = ZeroExt(1,x) + ZeroExt(1,y)
      + ZeroExt(256, cin)
s    = Extract(255, 0, wide)
cout = Extract(256, 256, wide)
```

Both 256-bit operands are zero-extended to 257 bits and added together with the zero-extension of the 1-bit carry; the low 256 bits give the result s, and the most significant bit is the carry output c_{out}. The encoding embeds carry propagation entirely into bit-vector arithmetic with no multi-return tuple or auxiliary predicate.

The same bit-vector semantics also covers the algebraic simplification of ordinary operators such as `add`, `sub`, `mul`, `and`, `or`, `xor`, and shifts; the carry-chain operator is the piece that a general-purpose IR lacks and that this paper adds.

Trusted base. The SMT equivalence check rests on the dMIR bit-vector semantic model and assumes that the model agrees with the runtime semantics of the actual JIT engine. On the 5884 differential tests of §4.3 no divergence has been observed; systematic consistency outside that distribution—model-versus-implementation fuzzing driven by random bytecode—is left as future work (§5.3).

3.2 Candidate Enumeration and SMT Admission

Admission check. For a candidate rule $LHS \to RHS$, we submit to Z3 the proposition $\exists\, \boldsymbol{v}.\ LHS(\boldsymbol{v}) \neq RHS(\boldsymbol{v})$. If Z3 returns *unsat*, the rule is equivalent on all inputs under the dMIR fixed-width modular bit-vector semantics and admission is granted; if Z3 returns *sat*, the solver also returns a counterexample and the rule is rejected; a timeout is treated as a rejection as well. The notion of equivalence here does not depend on two's complement, signed overflow, or any assumption beyond fixed-width modular arithmetic. All 70 production dMIR rules pass this check, the Z3 query scripts are committed alongside the rule files in the repository, and the continuous integration pipeline re-runs the checks on every rule change.

Two enumeration paths. Outside the existing rule set, a systematic candidate exploration was performed once to measure the capacity and false-miss rate of the synthesis pipeline. The two paths cover different shape spaces:

1. **Depth-2 algebraic enumeration**—with the dMIR arithmetic and Boolean operators as enumeration primitives, all left/right combinations of depth at most 2 are enumerated, covering the common algebraic identities.
2. **Carry-template enumeration**—multi-word carry-chain candidates are constructed by hand around the carry-chain operators.

Throughput and admission rate. The algebraic enumeration produces 775 candidates, of which 765 pass (admission rate **98.7%**; 10 rejected due to solver

timeout with no semantic counterexample); the carry templates produce 78 candidates, of which 17 pass (admission rate **21.8%**; the remaining 61 are semantic rejections). Full data with the Alive2 comparison appear in Table 3 in §4. The 61 semantic rejections among the carry templates illustrate that hand-written templates more easily reach the non-equivalence boundary—precisely where the SMT admission check is needed.

Relation between production rules and the synthesis pipeline. The 853 candidates above form a capability evaluation that is independent of the production rule set. The 70 dMIR production rules of this paper are designed by hand from runtime performance profiles and code review, and each is submitted to Z3 for bit-vector equivalence verification. The synthesis pipeline output serves as a Z3 admission-rate reference for future rule expansion and does not directly enter the production rule set. The 17 carry templates that pass the Z3 check are kept as a capability-evaluation result; they are not promoted to the production side because the production rules are driven by runtime performance profiles whose selection criterion differs from the input of the enumeration.

Counterexample examples. The following two carry templates rejected by Z3 illustrate why the SMT admission check is necessary on the carry chain. *Example 1: borrow discarded.* The candidate $\texttt{SBB}(x, x, c_f) \to 0$ has the counterexample $x = 0, c_f = 1$: the left-hand side evaluates to $\texttt{SBB}(0, 0, 1) = \texttt{0xFF...F}$ while the right-hand side is 0; the root cause is the ignored borrow input. The correct form $\texttt{SBB}(x, x, c_f) \to \texttt{sub}(0, c_f)$ passes the Z3 check. *Example 2: polarity error.* The candidate $\texttt{SBB}(x, 0, c_f) \to \texttt{sub}(0, c_f)$ has the counterexample $x = 1, c_f = 0$ (the left-hand side evaluates to 1 while the right-hand side is 0); the root cause is the dropped minuend, and the correct form is $\texttt{SBB}(x, 0, c_f) \to \texttt{sub}(x, c_f)$. Both examples reflect typical carry-chain reasoning traps under 256-bit unsigned modular arithmetic: a discarded carry input and confused operand polarity.

Artifact statement. The artifact contains the synthesis scripts (about 691 lines of Python), the rule-set JSON, and the Z3 query scripts.

3.3 Two-Tier Rule Set

Motivation for the split. The dMIR layer handles machine-independent algebraic simplification, and all 70 rules at this layer pass the Z3 equivalence check on the dMIR bit-vector model. The x86 CgIR layer handles hardware-specific redundancy cleanup on registers, flags, and branches; all 13 rules at this layer are mechanical syntactic rewrites and **currently have no separate SMT-check script**. Their correctness is covered by the matched test suites on the baseline and enabled sides of §4.3 (multipass JIT and interpreter unit tests plus Cancun state-transition tests in both execution modes, 5884 tests in total). A formal semantic model and SMT check for the x86 layer are listed as future work (§5.3).

Table 2. Classification of the two-tier peephole rewrite rules. The dMIR rules are checked for equivalence by Z3 on the bit-vector semantic model; the x86 CgIR rules are covered by the matched test suites.

Layer (verification)	Category	Count
dMIR (Z3 check)	Boolean normalization	42
	Algebraic identity	15
	Identity elimination	10
	Constant folding	2
	Dead-code elimination	1
	Subtotal	**70**
x86 CgIR (test coverage)	Redundant compare/test cleanup	8
	Branch-fallthrough cleanup	2
	No-op elimination	2
	test-setcc merge	1
	Subtotal	**13**
Total		**83**

Table 2 gives the classification of the two-tier rules. On the dMIR layer, Boolean-normalization rules form the largest family (42 rules, 60%), which is the family most widely covered by the algebraic enumeration. The algebraic-identity family (15 rules) contains classical identities such as add-zero elimination and multiply-one elimination. The identity-elimination family (10 rules) covers forms such as $\texttt{sub}(x, x) \to 0$ and $\texttt{and}(x, x) \to x$. The x86 layer is dominated by redundant compare and redundant test cleanup (8 rules in total), with 2 rules each for branch-fallthrough cleanup and no-operation elimination.

Complementarity. The two rule layers are complementary on real contracts. On U256-heavy contracts such as `weierstrudel`, the dMIR-layer rules dominate, whereas on the sha1 benchmarks the branch and compare cleanup at the x86 layer dominates. Table 4 in §4 gives the attribution data per rule family.

Asymmetric verification workflow. The 70 dMIR production rules are designed manually and submitted one by one to the Z3 equivalence check (§3.2); the synthesis pipeline is an independent capability-evaluation channel that does not participate in production rule maintenance; the x86-layer rules are covered by code review and the matched test suites. The limitations from this asymmetry are listed explicitly in §5.3.

4 Evaluation

This section evaluates the two-tier peephole rewrite system along three axes: the output of the rule synthesis pipeline against the general-purpose IR checker

Table 3. Output of the synthesis pipeline: candidate counts, Z3 passes, rejections, and admission rate. The two enumeration paths are described in detail in §3.2.

Candidate source	Input	Pass	Reject	Admission rate
Algebraic enumeration	775	765	10*	98.7%
Carry templates	78	17	61	21.8%

* 10 candidates returned neither sat nor unsat within the time limit and are conservatively rejected.

(§4.1), end-to-end execution performance (§4.2), and correctness with rule coverage (§4.3).

4.1 Rule Synthesis Output Against the General-Purpose IR

Table 3 combines the admission results from the synthesis pipeline of §3.2 with the verification ability of Alive2 on the same rule groups.

Z3 solves a typical 256-bit bit-vector peephole equivalence proposition in about 60 ms on average; all 70 dMIR rule queries finish within seconds, with no timeout or undecidable outcome, showing that the 256-bit width is not a bottleneck for the solver's capacity. From the synthesis pipeline, 10 typical U256 rules were sampled at random and compared against Alive2: 6 of them do not involve a multi-return carry structure and Alive2 discharges them directly; the other 4 involve `adc`/`sbb` multi-return structures, for which Alive2 either reports a type mismatch or times out (the mechanism is of the same type as Table 1 in §2.2, while these 4 are an independent sample from the production rule set), whereas Z3 discharges all of them directly under the dMIR bit-vector semantics, confirming the analysis of the IR expressiveness bottleneck in §2.2.

4.2 End-to-End Performance

Experimental setup and noise control. The baseline is the upstream main branch (which already contains 2 hard-coded x86 peephole rules that are not new in this paper); the enabled side adds the 83 new rules (70 dMIR + 13 x86). Both sides run through evmone-bench. Every benchmark is repeated 20 times, and the 95% confidence interval of the geometric mean is estimated by bootstrap ($B = 10\,000$). To suppress noise, the process is pinned to dedicated physical cores with `taskset -c 2-3` and raised to `nice -5`; measurement starts only when the 1-minute load average is below 0.5, with CPU turbo disabled; every benchmark is then re-measured to confirm that the coefficient of variation (CV) on both sides is below 3%, and if any benchmark exceeds the threshold, it is rerun with 30 repetitions.

Across the 27 benchmarks (Figure 3), the geometric-mean speedup of the enabled rule set over the baseline is **+7.24%** (95% confidence interval [+2.59%, +12.11%]), with a peak of **+25.79%** on `JUMPDEST_n0/empty`. The sha1 family

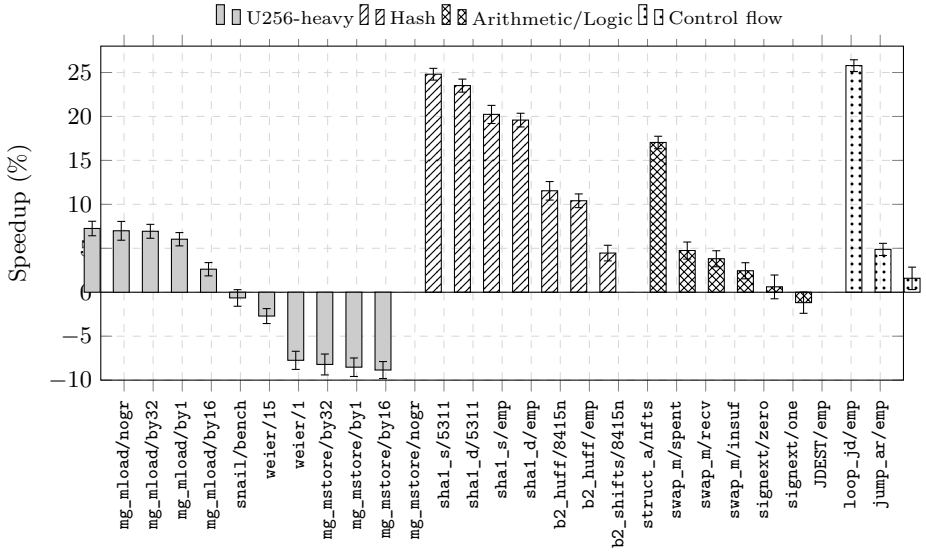

Fig. 3. Speedup on 27 benchmarks from enabling the peephole rewrite rule set over the disabled baseline (error bars are 95% confidence intervals). From left to right, the benchmarks are grouped into four clusters by workload type: U256-heavy (1–11), hash (12–18), arithmetic/logic (19–24), and control flow (25–27).

gains the most on long inputs: `sha1_shifts/5311` and `sha1_divs/5311` accelerate by **+24.81%** and **+23.51%** respectively, with the empty-input workloads still at **+19.59%**–**+20.24%**; `blake2b_huff` accelerates by **+10.41%**–**+11.54%**. Attribution by rule family appears in Table 4 (the hit rate is defined as the fraction of executed instructions matched by a given rule family): the U256-related rules reach almost 100% hit rate on the four `mg_mload` variants, 33.8% on `weierstrudel`, and 25.3% on `snailtracer`; the speedup on sha1 and `blake2b` comes mainly from the branch- and compare-merging rules at the x86 layer. Hit rate and speedup are not proportional: on `mg_mload` almost 100% of the instructions are matched by no-op elimination, yet the saving per rewrite is small, and the final speedup depends on both the trigger count and the cost of each rewrite. The two rule layers are complementary rather than substitutive: the multi-word rules yield concentrated gains on U256-heavy contracts, while the mechanical cleanup at the x86 layer provides broader, ordinary gains.

Discussion of the regressed surface. Five of the 27 benchmarks move beyond ±2% toward the slower side: the 4 variants of `memory_grow_mstore` are slower by **8.85%**, **8.21%**, **8.53%**, and **7.74%**, and `weierstrudel/1` is slower by **2.71%** (CV below 2% on both sides). The 4 `mstore` variants cluster on the same same-family write path and are not a diffuse disturbance (preliminary analysis in §5.3, item 7). As a control, the 4 `memory_grow_mload` variants of the same family accelerate by **+6.03%**–**+7.26%** under the same measurement recipe. In

Table 4. U256 rule hit distribution on representative benchmarks. The U256 hit rate is the number of instructions matched by U256 rules divided by the total number of executed instructions; benchmarks that have multiple same-family variants use the weighted mean.

Benchmark	U256 hit rate	Dominant rule family
mg_mload	99.9%	No-op elimination
mg_mstore	99.9%	No-op elimination*
weierstrudel	33.8%	Add-zero + no-op
snailtracer	25.3%	Dead-carry + no-op
sha1_divs/_shifts	7.5%	x86 branch/compare merge
jump_around†	0.0%	—

* Microbenchmark regression on the write path; see §5.3, item 7. † Control benchmark with no U256 instructions.

Table 5. Results on four differential test categories for the enabled side (83 new rules) and the baseline side (upstream main): passes/total. Identical numbers in both columns indicate that the enabled rule set introduces no correctness regression.

Test dimension	Enabled	Baseline	Suite source
Unit (JIT)	223/223	223/223	EVM spec subset
Unit (interpreter)	215/215	215/215	Run list
statetest (JIT)	2723/2723	2723/2723	Cancun 101 suites
statetest (interpreter)	2723/2723	2723/2723	Cancun 101 suites
Total	**5884/5884**	**5884/5884**	—

production-workload terms, the macro speedups on sha1 and `blake2b` (**+10%**–**+25%**) scale linearly with call count and input length, while the `mstore` regressions appear only in a microbenchmark of about 0.7 ns/op; the two magnitudes are not comparable.

Compilation overhead. A per-module timing switch was added to the EVMC adapter. Across the **776** modules of the JIT unit test set, the median single-module compilation time is **0.45 ms** and the 95th percentile is **0.87 ms**, covering the full pipeline of loading, analysis, and JIT code generation; dMIR peephole rewriting is only one local-matching pass within it. The 99th-percentile tail lands on the 19 large contracts that exceed 16 KB and correlates with contract size. The 0.87 ms at the 95th percentile is an upper bound on the wall-clock time of the entire compilation pipeline (loading, analysis, and JIT code generation combined), not the incremental cost of the dMIR peephole rewrite pass alone; per-pass baseline comparison is listed as future work in §5.3, item 6.

4.3 Correctness and Rule Coverage

The enabled side and the baseline side each run four differential test categories (Table 5); the two sides agree item-by-item on state and gas, for a cumulative

5884/5884. A pass on the baseline side shows that the test suite itself is correct; a pass on the enabled side shows that the 83 new rules introduce no correctness regression. The multipass JIT unit tests contain 223 independent Cancun cases; the interpreter run list contains 226 lines, reducing to 215 after the gtest deduplication; `evmone-statetest` on Cancun covers 101 suites (with `-k fork_Cancun` filtering out the Prague fork that DTVM does not yet support) and runs independently in both the multipass JIT mode and the interpreter mode. The cross-check between the two modes on the same test matrix produces identical outcomes, which is direct evidence of EVM consensus consistency.

Compared with the CAV 2023 rule set of Albert et al. [1], 7 of the 83 rules in this paper overlap: `dmir_add_zero`, `dmir_mul_one_rhs`, `dmir_mul_zero_rhs`, `dmir_or_zero`, `dmir_double_not`, `dmir_sub_self`, and `dmir_and_factor_lhs`, which correspond respectively to the CAV'23 rules `optimize_add_zero`, `optimize_mul_one`, `optimize_mul_zero`, `optimize_or_zero`, `optimize_not_not`, `optimize_sub_x_x`, and `optimize_and_and_l`. All 7 are general bit-vector algebraic identities ($\texttt{add}(x, 0) \to x$, $\texttt{mul}(x, 0) \to 0$, $\texttt{not}(\texttt{not}(x)) \to x$, and so on) that appear in almost every SMT-verifiable peephole effort. The remaining 76 rules cover dMIR algebraic simplification and multi-word cleanup (63 rules) and x86 register/branch/flag cleanup (13 rules). These transformations live at abstraction layers that the bytecode layer cannot reach directly—which is precisely the point that the two rule sets target different abstraction layers.

Continuous-integration auditability: the equivalence proposition and the Z3 query script of each dMIR rule are stored with the rule file and are re-verified automatically on every code change. Re-verifying all 70 dMIR rules takes about **0.5** seconds in total and is not a bottleneck; the 0.5 s figure is the total CI offline re-verification time and is not on the same scale as the runtime compilation latency in §4.2.

5 Related Work and Discussion

5.1 Verifiable Peephole Optimization Frameworks

This paper positions its work along three axes: the abstraction layer it targets, the verification method, and carry-chain coverage. Work on SMT-based verifiable peephole optimization traces back to Alive by Lopes et al. [5] and its successor Alive2 [4], and to Souper by Sasnauskas et al. [8]. Together, this line of work established the "synthesize candidates, then admit through SMT equivalence" paradigm. This paper follows the same paradigm, but changes the IR on which the check is performed from general-purpose LLVM IR to the dMIR of the EVM JIT. The PLDI 2015 Alive paper already points out that the multi-return encoding of carry and overflow flags in LLVM limits the expressiveness of peephole verification—this observation is the starting point of the present paper, not a new finding.

LeanMLIR by Bhat et al. at ITP 2024 [2] formalizes in Lean 4 the idea of "parameterizing the peephole verification framework over the IR", and its

positioning is complementary to ours: LeanMLIR focuses on the generality of the proof framework across IRs, whereas this paper focuses on the instantiated rule set and end-to-end integration on a concrete IR.

Hydra by Mukherjee and Regehr at OOPSLA 2024 [7] automatically discovers peephole rules that are independent of the input bit width. Like Alive2, however, it builds on LLVM IR, and therefore inherits the expressiveness limit on multi-word carry chains described in §2; the present paper offers one engineering solution by changing the underlying IR. The FMBC 2020 work of Schett and Nagele [9] uses a similar offline enumeration-plus-SMT pipeline to automatically generate about 993 rules for EVM *bytecode*; the equivalence guarantee is on par, and the difference is in the abstraction layer: the rule set of this paper acts on the JIT code-generation path and can exploit both structured matching on dMIR and mechanical cleanup on x86, while the gap between 993 and 83 rules reflects the difference in abstraction layer and coverage strategy. Adopting the bytecode-layer rule set as a JIT pre-processing stage and measuring the residual speedup after composition with the rules of this paper (an orthogonal-composition evaluation) is a separate design choice: the current DTVM production pipeline does not take the bytecode pre-processing path, so the present paper does not cover this combination and leaves it for future work.

The `adc/sbb` carry chain of multi-word integers is the standard implementation form used by GMP and libsecp256k1 on 64-bit hardware, and maps to the Intel ADCX/ADOX instructions; a carry chain of four 64-bit limbs is the standard lowered shape of EVM 256-bit arithmetic on 64-bit hardware.

5.2 Verification Work on the Consensus-Critical Path

Albert et al. at CAV 2023 [1] complete a Coq formal proof for an AOT optimization rule set on EVM bytecode blocks. Their positioning is bytecode-block level, offline-proof based, and built on general bit-vector identities; the positioning of this paper is at the JIT IR and machine-code layers, with SMT admission at rule synthesis time, and covers EVM-specific multi-word carry chains. The scales are not comparable: the 14 rules of Albert[1] all target stack-layer algebraic simplification; the 7 rules that overlap with our 83 are the algebraic-identity axioms (additive and multiplicative identities, multiplicative absorption, double negation, self-subtraction, absorption, etc.) that any SMT-verifiable bit-vector peephole system almost inevitably contains, and they do not imply a derivative dependency on the Albert rule set. Among the remaining 76 rules, 13 live at the x86 layer and 63 at the dMIR layer—covering the multi-word carry-chain rules that emerge from decomposing the 256-bit word, the `ADC/SBB` algebraic simplifications, and the constant propagation and strength reduction tied to the JIT code-generation path.

[1] The `forves` public artifact, stack-only release branch (Coq source `optimizations.v`, 14 rules) is used as the comparison baseline; the CAV 2023 paper text discusses 15 rules, and the extended-version Appendix A may list additional memory/storage-layer rules, but neither the present paper nor the Albert work touches those layers.

Runtime comparison-verification work—Breidenbach et al. at USENIX Security 2018, the Hydra Framework [3] (a different system from the OOPSLA 2024 Hydra above, sharing only the name), multi-client consensus-divergence detection systems, EVM fuzz testing, and consensus test networks—detects divergence after deployment; the present paper, in contrast, positions itself as static hardening during development.

5.3 Limitations and Future Work

1. **Trusted base.** The SMT check rests on the dMIR bit-vector semantic model and assumes its agreement with the runtime semantics of the actual JIT engine; systematic testing of that agreement is future work.
2. **x86-layer verification coverage.** All 13 x86 rules are mechanical syntactic rewrites without a separate SMT check. Their correctness is guaranteed by the matched test suites of §4.3; a formal model of x86 register and flag semantics is future work.
3. **Workload coverage.** The speedup figures come from the 27 evmone-bench benchmarks and do not represent all production contracts.
4. **Scope of rule modeling.** Only the arithmetic and flag semantics before and after each peephole rewrite are checked for equivalence; gas accounting, memory aliasing, and storage-slot side effects are not modeled.
5. **Dead-carry elimination is ineffective at the head of U256 add/sub chains.** At the head of a U256 add/sub chain, the third operand of the carry-propagating addition is a runtime value rather than a matchable zero constant, so the rule has no effect there by construction; the rule is kept because it remains effective on the independent carry-propagating additions inside the U256 multiplication lowering.
6. **Measurement precision and compilation-overhead comparison.** `weierstrudel/15` and `signextend/one` fall within the ±2% noise band; `weierstrudel/1` stably slows down by **2.71%**, and disabling each new rule in turn revealed no consistent negative contribution. The median compilation time of 0.45 ms and the 95th percentile of 0.87 ms per module are single-side measurements on the enabled build; the EVMC interface has no per-stage timing hook, a baseline comparison needs separate instrumentation, and this paper only reports an upper bound on the overall compilation latency.
7. **Microbenchmark regression on the `memory_grow_mstore` family.** The 4 `mstore` variants slow down stably by **7.74%**–**8.85%** (CV below 2% on both sides). A preliminary analysis localizes the regression to the matching behavior of the x86-layer compare-merging rule `optimizeCmp` on the memory-expand control-flow lowering path—that path lowers as a `SETCC`→ zero-extend→`TEST`→`JCC` chain with the zero extension using `SUBREG_TO_REG`, which does not exactly match the compact chain covered by the current rule set; the exact root cause is left for future work. The 4 `mload` variants of the same family do not expose this merging opportunity and accelerate by **+6.03%**–**+7.26%**.

6 Conclusion

This paper builds a two-tier peephole rewrite system for the deterministic EVM JIT: the 70 dMIR rules pass the Z3 bit-vector equivalence check, and the 13 x86 rules are guaranteed by the matched test suites. A small intersection with the bytecode block-level rule set of Albert et al. at CAV 2023 [1] consists of general bit-vector identities, and the rest is complementary. The methodology—modeling carry-propagating add/sub as first-class bit-vector operators in a domain-specific IR so that automatic equivalence checking covers a larger surface—applies in principle to other deterministic execution environments with multi-word arithmetic, such as the WebAssembly big-integer extensions and zk-VM back-ends.

Data and Code Availability

The rule synthesis scripts, rule sets (JSON and Z3 query files), evaluation scripts, and experimental data are released to reviewers through an anonymous mirror; the URL is provided in the double-blind submission materials and will become a de-anonymized public release upon acceptance. The equivalence check of each rule is re-executed automatically by the continuous-integration pipeline whenever the rule file changes.

References

1. Albert, E., Genaim, S., Kirchner, D., Martin-Martin, E.: Formally verified EVM block-optimizations. In: Computer Aided Verification (CAV). LNCS, vol. 13966, pp. 176–189 (2023). https://doi.org/10.1007/978-3-031-37709-9_9
2. Bhat, S., Keizer, A., Hughes, C., Goens, A., Grosser, T.: Verifying peephole rewriting in SSA compiler IRs. In: 15th International Conference on Interactive Theorem Proving (ITP). LIPIcs, vol. 309, pp. 9:1–9:20 (2024). https://doi.org/10.4230/LIPIcs.ITP.2024.9
3. Breidenbach, L., Daian, P., Tramèr, F., Juels, A.: Hydra: Principled bug bounties for smart contract security. In: 27th USENIX Security Symposium (USENIX Security 18) (2018)
4. Lopes, N.P., Lee, J., Hur, C.K., Liu, Z., Regehr, J.: Alive2: Bounded translation validation for LLVM. In: Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation (PLDI) (2021). https://doi.org/10.1145/3453483.3454030
5. Lopes, N.P., Menendez, D., Nagarakatte, S., Regehr, J.: Provably correct peephole optimizations with alive. In: Proceedings of the 36th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI) (2015). https://doi.org/10.1145/2737924.2737965
6. de Moura, L., Bjørner, N.: Z3: An efficient SMT solver. In: Tools and Algorithms for the Construction and Analysis of Systems (TACAS) (2008). https://doi.org/10.1007/978-3-540-78800-3_24

7. Mukherjee, M., Regehr, J.: Hydra: Generalizing peephole optimizations with program synthesis. Proc. ACM Program. Lang. **8**(OOPSLA1) (2024). https://doi.org/10.1145/3649837
8. Sasnauskas, R., Chen, Y., Collingbourne, P., Ketema, J., Taneja, J., Regehr, J.: Souper: A synthesizing superoptimizer (2017)
9. Schett, M.A., Nagele, J.: Populating the peephole optimizer of a smart contract compiler. In: 2nd Workshop on Formal Methods for Blockchains (FMBC 2020). OASIcs (2020). https://doi.org/10.4230/OASIcs.FMBC.2020.3